TD 07: Sémantique à grand pas

1 Noyau OCaml

On considère le noyau OCaml suivant, avec sa sémantique à grand pas.

EXPRESSION

$e ::=$	x	(variable)
	n	(nombre littéral)
	$e_1 + e_2$	(addition)
	`let` x`=`e_1 `in` e_2	(définition)

Priorité :
(définition) <(addition)

Associativité à gauche :
addition

Lieur :
définition : x dans e_2

Valeurs $v ::= n \mid x$.

$$\textsc{Val}\ \frac{}{v \Downarrow v} \qquad \textsc{Add}\ \frac{e_1 \Downarrow n_1 \qquad e_2 \Downarrow n_2 \qquad n = n_1 + n_2}{e_1 + e_2 \Downarrow n} \qquad \textsc{Let}\ \frac{e_1 \Downarrow v_1 \qquad e_2\{v_1/x\} \Downarrow v_2}{\texttt{let}\ x\texttt{=}e_1\ \texttt{in}\ e_2 \Downarrow v_2}$$

Question 1 – Évaluer les expressions suivantes. Dessiner l'arbre de dérivation.
- `let` x=10 `in` $x+1$.
- `let` x=(`let` y=1 `in` $9+y$) `in` $x+1$

2 Expressions booléennes

On considère le calcul d'expressions booléennes suivant.

EXPRESSIONS

$b ::=$	`true`	(vrai)
	`false`	(faux)
	b_1 `&&` b_2	(et)
	b_1 `||` b_2	(ou)
	`not` b	(negation)

Priorité : (negation) > (et) > (ou)

Associativité gauche : et, ou

On rappelle le principe de l'évaluation des expressions booléennes de gauche à droite de manière paresseuse :
- si le résultat de l'évaluation de l'expression de gauche d'un (et) est (faux), l'expression de droite n'est pas évaluée, et le résultat du (et) est (faux) ;
- si le résultat de l'évaluation de l'expression de gauche d'un (ou) est (vrai), l'expression de droite n'est pas évaluée, et le résultat du (ou) est (vrai).

Question 2 – Définir l'évaluation paresseuse de ce langage sous forme de sémantique à grand pas.

3 Exceptions

On rappelle la syntaxe et sémantique à petit pas du λ-calcul avec exceptions.

TYPE

$$\begin{array}{rll} \tau ::= & \mathtt{str} & \text{(type des chaînes)} \\ & \tau_1 \to \tau_2 & \text{(type des fonctions)} \end{array}$$

EXPRESSION

$$\begin{array}{rll} e ::= & x & \text{(variable)} \\ & s & \text{(chaîne littérale)} \\ & e_1 \mathbin{\hat{}} e_2 & \text{(concaténation)} \\ & e_1\ e_2 & \text{(application)} \\ & \mathtt{fun}\ (x{:}\tau) \Rightarrow e & \text{(abstraction)} \\ & \mathtt{raise}\ e & \text{(déclencheur)} \\ & \mathtt{try}\ e_1\ \mathtt{with}\ x.e_2 & \text{(récupération)} \end{array}$$

Priorité :

(récupération) < (abstraction)
< (déclencheur)
< (concaténation)
< (application)

Associativité à gauche :
concaténation, application

Associativité à droite :
type des fonctions

Lieurs :
abstraction : x dans e
récupération : x dans e_2

APPD
$$\frac{e_2 \to e_2'}{e_1\ e_2 \to e_1\ e_2'}$$

APPG
$$\frac{e_1 \to e_1'}{e_1\ v \to e_1'\ v}$$

BETA
$$\frac{}{(\mathtt{fun}\ (x{:}\tau) \Rightarrow e)\ v \to e\{v/x\}}$$

TRY
$$\frac{e_1 \to e_1'}{\mathtt{try}\ e_1\ \mathtt{with}\ x.e_2 \to \mathtt{try}\ e_1'\ \mathtt{with}\ x.e_2}$$

TRYV
$$\frac{}{\mathtt{try}\ v\ \mathtt{with}\ x.e_2 \to v}$$

TRYR
$$\frac{}{\mathtt{try}\ (\mathtt{raise}\ v)\ \mathtt{with}\ x.e_2 \to e_2\{v/x\}}$$

CONCATG
$$\frac{e_1 \to e_1'}{e_1 \mathbin{\hat{}} e_2 \to e_1' \mathbin{\hat{}} e_2}$$

CONCATD
$$\frac{e_2 \to e_2'}{s_1 \mathbin{\hat{}} e_2 \to s_1 \mathbin{\hat{}} e_2'}$$

CONCAT
$$\frac{s_1 \mathbin{\hat{}} s_2 = s}{s_1 \mathbin{\hat{}} s_2 \to s}$$

RAISE
$$\frac{e \to e'}{\mathtt{raise}\ e \to \mathtt{raise}\ e'}$$

RAISEAPPD
$$\frac{}{e\ \mathtt{raise}\ v \to \mathtt{raise}\ v}$$

RAISEAPPG
$$\frac{}{(\mathtt{raise}\ v_1)\ v_2 \to \mathtt{raise}\ v_1}$$

CONCATRAISEG
$$\frac{}{\mathtt{raise}\ v \mathbin{\hat{}} e_2 \to \mathtt{raise}\ v}$$

CONCATRAISED
$$\frac{}{s_1 \mathbin{\hat{}} \mathtt{raise}\ v \to \mathtt{raise}\ v}$$

RAISERAISE
$$\frac{}{\mathtt{raise}\ (\mathtt{raise}\ v) \to \mathtt{raise}\ v}$$

Question 3 – Donner la sémantique à grand pas du λ-calcul avec exceptions.